MEETMIND

AI Meeting Action-Item Extractor

Course Project Report

Project type	AI / Natural Language Processing prototype
Interface	Python + Streamlit
Output	Editable action items, CSV and JSON

Prepared as a prototype report. Add your name, course details, screenshots, and actual test observations before submission.

Abstract

MeetMind is a small application that turns meeting notes or transcript text into a structured list of likely action items. For each item, it attempts to identify the task, owner, deadline, status, and confidence. It flags incomplete records for human review and lets users edit and export the results. The application provides an offline rule-based mode for demonstrations and an optional AI-model API mode for more flexible extraction.

1. Introduction

Meetings often include decisions and follow-up tasks mixed with general discussion. People may forget to record who is responsible or when work is due. MeetMind aims to make follow-up easier by organizing likely action items from text into a table.

2. Problem Statement

Manually reading meeting notes and recording tasks can be time-consuming. Important action items, owners, and deadlines may be overlooked. A lightweight text extraction tool can help identify candidate tasks and make missing information visible.

3. Objectives

• Accept meeting notes or transcript text.
• Detect likely action-item sentences.
• Extract task, owner, deadline, status, and confidence.
• Flag records with missing owner or deadline.
• Allow the user to review and edit results.
• Export action items in CSV and JSON formats.
• Offer an optional AI API mode alongside an offline mode.

4. Scope

The prototype accepts pasted text. It does not record meetings or transcribe audio. It is intended as an assistive tool: users must verify the output before assigning or tracking tasks.

5. Tools and Technologies

• Python: application logic and text processing.
• Streamlit: browser-based user interface.
• Pandas: structured result table and file export.
• Regular expressions: offline detection of common task, owner, and deadline patterns.
• Optional language-model API: structured extraction from a model when a valid API key and network connection are available.

6. System Requirements

Software: Python 3.10 or newer, pip, Streamlit, Pandas, Requests, and a modern web browser. API mode additionally requires internet access and a valid API key.
Hardware: A standard laptop or desktop capable of running Python and a browser.

7. Methodology

1. The user pastes meeting notes into the text area.
2. The user selects Offline smart rules or AI model via API.

3. In offline mode, the application splits text into sentences/lines and looks for action verbs and commitment phrases.
4. The application checks common formats for owners and deadlines.
5. In API mode, the transcript is sent to the selected language model with instructions to return structured JSON. This mode requires a valid key and may incur usage charges.
6. Results are shown in an editable table.
7. The user reviews uncertain fields and downloads the result as CSV or JSON.

8. System Design

Input: Meeting transcript or notes.
Processing: Sentence splitting, action detection, owner/deadline extraction, confidence assignment, or optional model-based structured extraction.
Output: Editable action-item table with Task, Owner, Deadline, Status, Confidence, and Needs review fields, plus CSV/JSON downloads.

9. Sample Input

Aisha will prepare the presentation slides by Friday.
Rahul will test the login and signup flow tomorrow.
The team discussed the project budget and launch risks.
Assigned to Priya: send the final project report by October 15.

10. Expected Output

The application should identify likely tasks such as preparing slides, testing the login flow, and sending the report. It should try to identify owners and deadlines and exclude ordinary discussion where no action is assigned. Results may vary by extraction mode and wording; the user should verify all fields.

11. Testing Plan

Run these manual tests and record the actual outcome after execution:
1. Task with a clear owner and deadline.
2. Task with an owner but no deadline.
3. Task with a deadline but no owner.
4. General discussion without a task.
5. Duplicate task sentence.
6. Export the result to CSV and open it.
7. If API mode is used, test a valid response and an invalid/missing key.

12. Limitations

Offline mode is rule-based and is not a trained AI model. It may miss unusual phrasing, indirect commitments, or complex date expressions. API mode requires connectivity, a valid key, model access, and may cost money. The application does not guarantee correct task extraction and requires human review.

13. Future Scope

- Add upload support for TXT, DOCX, and PDF transcripts.
- Integrate audio-to-text transcription.
- Improve date normalization and named-entity recognition.
- Add a database, task status tracking, and reminders.

• Add user authentication and team collaboration.

14. Conclusion

MeetMind demonstrates a practical workflow for turning meeting text into structured follow-up tasks. Its editable results, review flags, and exports can reduce manual organization work. The optional model-based mode offers more flexible extraction, while the offline mode supports a simple demonstration without external credentials. Human review remains essential.

Appendix: Manual Test Cases

ID	Test	Expected behavior
TC01	Aisha will send the report by Friday.	Likely task; owner Aisha; deadline Friday.
TC02	Prepare the demo slides.	Likely task may appear; missing fields flagged.
TC03	The team discussed the budget.	Should not be treated as an assigned task.
TC04	Repeat an identical action sentence.	Duplicate should be removed in offline mode.
TC05	Download results as CSV.	CSV downloads and opens as a table.
TC06	API mode with no key.	Error message asks user to enter a key or switch modes.

Important: replace expected behavior with actual results and add screenshots after running the program. Do not report unperformed tests as completed.